Jayen Thakker
Data Analytics Mentor

2025

Data Analytics

Data Cleaning Checklist

Jayen Thakker
Data Analytics Mentor

👆 Swipe to continue reading

Data Cleaning with Excel

1. Remove Duplicates

Go to:
Data → Remove Duplicates

✅ Helps clean repeated rows.
Useful when combining datasets or cleaning exported CRM data.

2. Trim Extra Spaces

Use:
`=TRIM(A1)`

✅ Removes unnecessary spaces before /after text.
Perfect for cleaning names or email addresses.

3. Fix Inconsistent Text Case

Use:

- `=UPPER(A1)` – All caps
- `=LOWER(A1)` – All small
- `=PROPER(A1)` – Capitalize first letter

✅ Great for standardizing entries like city names or product titles.

4. Find and Replace

Go to:
Home → Find & Select → Replace

✅ Replace missing values like "n/a", "null", or "-" with blank cells or "NA".

5. Handle Missing Data

- Use filters to locate blanks
- Use formulas like:
  `=IF(A1="", "Missing", A1)`

✅ Ensures consistency in analysis.

6. Split Data into Columns

Use:
Data → Text to Columns

✅ Breaks combined data like "FirstName LastName" or "City, State".

7. Use Flash Fill

Shortcut: `Ctrl + E`

Predicts and fills patterns—great for splitting or merging columns without formulas.

8. Convert Numbers Stored as Text

Select the column → ⚠️ warning icon → Convert to Number

✅ Ensures numerical operations (like SUM, AVERAGE) work correctly.

9. Data Validation

Go to:
Data → Data Validation

✅ Prevents invalid entries (like text in a phone number field).
Add dropdowns for standard values.

10. Remove Blank Rows

Select the row → Right-click → Delete

Or use filters to isolate and delete blank rows in bulk.

Jayen Thakker
Data Analytics Mentor

Swipe to continue reading

Data Cleaning with Excel vs Python vs SQL

Task	Excel	Python (Pandas)	SQL
Ease of Use	Very easy, UI-based, beginner-friendly	Requires coding skills	Moderate, needs SQL knowledge
Handling Large Data	Limited (up to ~1M rows)	Excellent, handles millions of rows efficiently	Excellent, works directly with large databases
Removing Duplicates	Built-in "Remove Duplicates" button	`df.drop_duplicates()`	`SELECT DISTINCT` or `ROW_NUMBER()` for deduplication
Handling Null/Missing Values	Replace with value, filter, or delete manually	`df.fillna()`, `df.dropna()`	`IS NULL`, `COALESCE()`, or filtering with `WHERE`
Data Type Conversion	Manual, with functions like `TEXT()`, `VALUE()`	`df.astype()`	`CAST()` or `CONVERT()`
Trimming Whitespace	`TRIM()` function	`df['col'].str.strip()`	`TRIM(column)`
Splitting/ Combining Columns	`TEXT TO COLUMNS`, `&`, `CONCATENATE()`	`str.split()`, `+`, `agg()`	`SUBSTRING()`, `CONCAT()`, `SPLIT_PART()`
Filtering Data	Filter buttons, conditional formatting	`df[df['col'] == value]`	`WHERE`, `IN`, `LIKE`
Finding & Replacing	Find & Replace tool (`Ctrl + H`)	`str.replace()`	`REPLACE()`, `UPDATE SET`
Outlier Detection	Manual visual inspection or formulas	Z-score, IQR methods via code	Limited, requires advanced queries
Automation / Reusability	Limited (macros or scripts needed)	Highly reusable scripts and notebooks	Reusable via stored procedures or views
Visualization for Cleaning	Built-in charts, conditional formatting	Matplotlib, Seaborn, Pandas profiling	Limited, usually exported to BI tools
Best For	Small datasets, one-time quick fixes	Complex, repeatable, and automated workflows	Cleaning data within databases before analysis

Jayen Thakker
Data Analytics Mentor

Swipe to continue reading

6 Steps of data cleaning every data analyst should know

1 Explore the Dataset

- Identify **data sources**.
- Understand data types (**numeric**, **categorical**).
- Spot potential **data quality issues.**

Key Actions

- Review columns & value ranges.
- Define the dataset's purpose.

2 Handle Missing Data

- **Missing values** can distort analysis & must be **addressed** effectively.

Key Actions

- Identify missing values (e.g., **blanks**, **NULLs**).
- **Remove rows/columns** with many missing values.
- Impute using **mean**, **median**, or **mode**.

3 Remove Duplicates

- **Duplicates** can skew analysis and inflate results.

Key Actions

- Check for **identical rows** or **records**.
- **Remove duplicate** entries while retaining unique data.

4 Handle Formatting

- **Inconsistencies** in formatting can lead to **errors in analysis.**

Key Actions

- Standardize date formats (e.g., **DD-MM-YYYY** or **MM/DD/YYYY**).
- Align text capitalization (e.g., **"Male"** vs. **"male"**).
- Ensure uniform units (e.g., converting **km** to **miles**).

5 Managing Outliers

- **Outliers** can indicate **errors** or **meaningful insights.**

Key Actions

- Use statistical methods (e.g., **Z-score**, **IQR**) to identify outliers.
- Decide whether to **remove**, **cap**, or **analyze outliers** further.

6 Validating Data

- The final step ensures that the **cleaned dataset** is **accurate** & ready for analysis.

Key Actions

- **Cross-verify cleaned data** with the original source.
- **Validate** against predefined data quality rules.
- **Test data** for **consistency** and **reliability** in the analysis.

Jayen Thakker
Data Analytics Mentor

Swipe to continue reading

6 Ways to Remove Duplicates in SQL

1 Using DISTINCT

```
SELECT DISTINCT
    column1, column2, ...
FROM your_table;
```

Key Actions

- Use this when you don't **need to delete** anything from the table, just want a **deduplicated result.**

2 Using GROUP BY

```
SELECT column1, column2,
MIN(id) AS min_id
FROM your_table
GROUP BY column1, column2;
```

Key Actions

- **Groups duplicates** and returns **a single row per group** using **MIN**, **MAX**, etc.

3 Using ROW_NUMBER()

```
SELECT *, ROW_NUMBER() OVER (
  PARTITION BY column1,
column2
  ORDER BY id
) AS rn
FROM your_table;
```

Key Actions

- Assigns a row number within each duplicate group.

4 DELETE with CTE

```
WITH CTE AS (
  SELECT *,
  ROW_NUMBER() OVER (
    PARTITION BY
        column1, column2
    ORDER BY id
  ) AS rn
  FROM your_table
)
DELETE FROM CTE WHERE rn > 1;
```

Key Actions

- **Deletes duplicates** while keeping the first entry from each group.

5 Back up before deleting

```
CREATE TABLE
    your_table_backup AS
  SELECT * FROM your_table;
```

Key Actions

- Always **create a backup** before running any **delete operation.**
- Gives you a **safety net** in case something goes wrong.

6 Using a temp table to rebuild deduplicated data

```
CREATE TABLE temp_table AS
SELECT DISTINCT *
FROM your_table;
```

```
DROP TABLE your_table;
```

```
ALTER TABLE temp_table
RENAME TO your_table;
```

Key Actions

- Replace the **original table** with a **deduplicated version safely.**

Jayen Thakker
Data Analytics Mentor

Swipe to continue reading

How to Clean Data with SQL?

A Step-by-step guide

A structured approach to cleaning raw datasets using SQL, covering:

- Removing duplicates
- Handling NULL values
- Standardizing Data Formats
- Fixing Incorrect Data
- Removing Unwanted Characters
- Handling Inconsistent Categorizations

Master SQL data cleaning techniques, one query at a time!

Jayen Thakker
@jayenthakker

👆 Swipe to continue reading

Step 1

Remove Duplicates

Duplicate records can inflate calculations and distort results. Use DISTINCT to check for duplicates and DELETE to remove them.

✅ Finding duplicate rows

```
SELECT column1, column2, COUNT(*)
FROM table_name
GROUP BY column1, column2
HAVING COUNT(*) > 1;
```

✅ Removing duplicates while keeping one record

```
DELETE FROM table_name
WHERE id NOT IN (
    SELECT MIN(id)
    FROM table_name
    GROUP BY column1, column2
);
```

Jayen Thakker
@jayenthakker

👆 Swipe to continue reading

Step 2

Handling Missing Values

Missing values can break calculations. You can either remove, replace, or infer them.

✅ Identifying missing values

```
SELECT * FROM table_name
WHERE column1 IS NULL;
```

✅ Replacing NULL values

Replace NULLs with a default value using **COALESCE**.

```
UPDATE table_name
SET column1 = COALESCE(column1, 'Default Value');
```

For numerical columns, you can replace NULLs with the **mean** or **median.**

```
UPDATE table_name
SET column1 =
     (SELECT AVG(column1)
      FROM table_name
      WHERE column1 IS NOT NULL)
WHERE column1 IS NULL;
```

Jayen Thakker
@jayenthakker

👆 Swipe to continue reading

Step 3

Standardizing Data Formats

Inconsistent formats can create errors in joins, filters, and calculations.

✅ Converting text to lowercase

```
UPDATE table_name
SET column1 = LOWER(column1);
```

✅ Trimming unnecessary spaces

```
UPDATE table_name
SET column1 = TRIM(column1);
```

✅ Standardizing date formats

```
UPDATE table_name
SET date_column = STR_TO_DATE(date_column, '%Y-%m-%d');
```

Jayen Thakker
@jayenthakker

👆 Swipe to continue reading

Step 4

Fixing Incorrect Data

Check for outliers and incorrect values.

✅ Finding outliers in numerical data

```
SELECT *
FROM table_name
WHERE column1 > (SELECT AVG(column1) + 3 * STDDEV(column1)
FROM table_name);
```

✅ Replacing incorrect values

```
UPDATE table_name
SET column1 = 'CorrectValue'
WHERE column1 = 'IncorrectValue';
```

Step 5

Removing Unwanted Characters

✅ **Use REPLACE() to clean unwanted symbols.**

```
UPDATE table_name
SET column1 = REPLACE(column1, '$', '');
```

✅ **For advanced cleaning, use REGEXP_REPLACE() in MySQL 8+ and PostgreSQL.**

```
UPDATE table_name
SET column1 = REGEXP_REPLACE(column1, '[^0-9]', '');
```

Jayen Thakker
@jayenthakker

👆 Swipe to continue reading

Step 6

Handling Inconsistent Categorizations

Standardize categorical values to avoid duplicate groupings.

✅ Checking distinct values

```
SELECT DISTINCT column1 FROM table_name;
```

✅ Merging similar values

```
UPDATE table_name
SET column1 = 'United States'
WHERE column1 IN ('US', 'U.S.', 'USA');
```

Jayen Thakker
@jayenthakker

BECOME DATA ANALYTICS SKILL-READY IN 90 DAYS

Want a **Data Analytics Career** but unsure where to begin?

My proven **90-day Roadmap** gets you job-ready:

- **Phase 1 (Days 1-30)**
  → SQL & Excel
- **Phase 2 (Days 31-60)**
  → Data Visualization Concepts & Tableau/Power BI
- **Phase 3 (Days 61-90)**
  → Hands-on Projects & Job Prep

No confusion. Just clarity and action.

Comment "READY"
I'll guide you instantly.